CareerConnect - Sprint 2 AI Use Log

Entry number: 1

Date: 2026-10-20

AI tool/model: ChatGPT

Task ID/Title: Task #7.5 / #124 - Applicant Form Interface and Navigation

Purpose of AI Use: To implement a dedicated application form that opens from job details, includes the requested applicant fields, and uses the existing submission functionality without disrupting teammates' code.

Chat Link or Prompt/Response: See Appendix

AI-Suggested Content: AI provided a full React/Next.js client form with personal information, address, education, experience, skills, languages, professional links, resume selection, and an optional cover-letter field. It also provided a Next.js dynamic job application route, a navigation link from the job-details panel, confirmation dialog, and feedback messages. It advised retaining the teammate-owned Supabase submission action and noted that the existing backendonly saved the job application and resume.

Validation: Ran ESLint and the Next.js production build; corrected the accidentally named dynamic route folder; confirmed that selecting a job and clicking Apply opened the dedicated application page with the correct job title and company.

Decision: Accepted with limitation - UI and navigation implemented; expanded form fields and written cover letter were not yet persisted.

Reflection: AI helped structure a large form and identified the separation between a functioning UI and stored application data. I verified the implementation with actual project build and browser checks.

Responsible Person: Daksikan Navanithan

Entry number: 2

Date: 2026-10-20

AI tool/model: ChatGPT

Task ID/Title: Task #7.5 / #124 - Import Saved Profile Data

Purpose of AI Use: To reuse existing profile information in the application form and resolve TypeScript errors in the profile-import server action.

Chat Link or Prompt/Response: See Appendix

AI-Suggested Content: AI identified the existing profile retrieval functions and suggested an application-specific server action to collect personal information, education, work experience, skills, and languages. It mapped database fields to form fields, converted dates to YYYY-MM, supported custom degrees, and wired a Fill from profile button with a confirmation before overwriting data. After TypeScript errors

involving the optional languages property, AI revised the code to narrow the result type using a property check and a helper function.

Validation: Checked the exported profile functions and return shapes; ran ESLint and Next.js build after revisions. The final reported build completed TypeScript, page generation, and route checks successfully.

Decision: Accepted - profile import code integrated and type-checked. Runtime behavior should be tested with saved profile data.

Reflection: The guidance was useful for learning to reuse existing server functions rather than duplicating backend logic. The repeated language typing error also showed why inspecting real return types is important.

Responsible Person: Daksikan Navanithan

Entry number: 3

Date: 2026-10-22

AI tool/model: ChatGPT

Task ID/Title: Task #7.5 / #124 - Already-Applied Detection and User Feedback

Purpose of AI Use: To notify users immediately when they have already applied for a job and prevent them from seeing an unnecessary second form.

Chat Link or Prompt/Response: See Appendix

AI-Suggested Content: AI proposed checking the applications table on the server using the authenticated applicant ID and selected job ID. It supplied a revised application page with three outcomes: display the regular form when no prior application exists, display an Already Applied panel with status and navigation if one exists, or show an error when the lookup fails. AI advised preserving the database unique constraint as a final duplicate-protection layer. It also helped correct a JSX apostrophe lint error.

Validation: Re-ran ESLint and confirmed no errors after correcting the JSX text. Tested the previously applied-to job in the browser and confirmed the Already Applied message appeared correctly.

Decision: Accepted - duplicate-application detection and browser behavior verified.

Reflection: The solution improved the user experience without changing the existing submission backend and highlighted that server-side UI checks supplement, rather than replace, database protections.

Responsible Person: Daksikan Navanithan

Appendix - Prompts and AI Responses

Entry 1 - Applicant form and dedicated application page

AI prompt (original request with context): "can you send me the revised script" - referring to the Task #124

JobApplicationForm.tsx and the separate job application page after discussing a seven-section application form and the

Apply button in JobDetails.tsx.

AI response : Replace the JobApplicationForm component with a client-side form containing seven sections: personal information; address; education; work experience; skills and languages; professional links; and application documents. Use existing getActiveApplicationResumes and submitJobApplication server actions, provide an Upload a new resume link, confirmation modal, and success/error messages. In JobDetails.tsx, replace inline ApplyToJob usage with a Next.js Link pointing to /jobs/[jobID]/apply. On the server page, authenticate the user and fetch the open job to display its title, company and location. Be explicit that additional applicant details are not saved by the current submission backend.

Implementation / follow-up: The build initially failed because the folder contained literal PowerShell backticks around [jobID]. Renaming it to [jobID] fixed route discovery. ESLint and the production build then passed.

Entry 2 - Profile autofill and TypeScript correction

AI prompt (original request with context): "revise the script" and "revise and send right one" - referring to profile-import-actions.ts after errors in languagesResult.languages and implicitly typed map/filter callbacks.

AI response: Reuse getCurrentProfile, getCurrentEducation, getCurrentExperience, getCurrentSkills, and getCurrentPreferenceLanguages. Return normalized form data, map full_name into editable first/last names, convert education and experience dates to month values, and import skills and languages. Because the preferences function may return an error object with no languages property, extract languages in a helper that narrows the result with "languages" in result before accessing the property. In the form, enable Fill from profile, ask before overwriting entered data, handle Other degree values, and keep manually entered phone number intact. No teammate profile backend changes are required.

Implementation / follow-up: The corrected version subsequently passed ESLint, TypeScript checks, and production build. The transcript documents this build result, not a definitive end-to-end test of populated profile fields.

Entry 3 - Existing application warning

AI prompt (original): "it doesnt show that i already applied for that job". The user later supplied the complete src/app/jobs/[jobID]/apply/page.tsx and asked for the revised file.

AI response: Query Supabase applications with job_id equal to the selected job and applicant_id equal to the authenticated user. If an existing application is found, show an Already Applied panel, its current status when available, a View my applications link, and a Back to job listings link instead of the form. On query failure, show a verification error rather than assuming no application exists. Keep the submission function's database uniqueness check for race-condition protection. The query belongs in page.tsx, not in a PowerShell command.

Implementation / follow-up: The initial revised JSX contained apostrophes that triggered react/no-unescaped-entities. Replacing "couldn't" / "you've" with "could not" / "you have" resolved lint. The user confirmed that the already-applied message worked in the browser.